CLOUD COMPUTING & AWS EC2

Assignment / Practical Task

Submitted by: Sujal Chouhan

Subject: Cloud Computing

Topics: IaaS, PaaS, SaaS, AWS EC2, SSH and Static Website Hosting

Note: Task 3 and Task 4 require actions inside your own AWS account. This PDF gives the procedure and places to record your actual screenshot and command output; it does not fabricate account evidence.

Task 1: Classify Three Popular Apps

I use the following apps as examples. These are user-facing applications, so they are most similar to Software as a Service (SaaS).

App	Closest model	Reason
Instagram	**SaaS**	It provides a ready-to-use social media service through an app or website. Users do not manage the servers or platform.
Zomato	**SaaS**	It provides ready-made restaurant discovery, menus and food-ordering features. The user only uses the service.
Spotify	**SaaS**	It provides music streaming through an app or website without users managing the underlying infrastructure.

Note: The apps may use IaaS or PaaS internally, but from the customer's point of view they are SaaS products.

Task 2: Comparison of IaaS, PaaS and SaaS

Point	IaaS	PaaS	SaaS
Meaning	Infrastructure as a Service	Platform as a Service	Software as a Service
What you get	Virtual machines, storage and networking	Managed environment to build, test and deploy apps	A complete application ready to use
Customer manages	OS, apps, data and configurations; provider manages physical hardware	Application code and data; provider manages much of the platform and infrastructure	Mainly account, settings and own content/data
Typical user	System administrator or infrastructure team	Software developer	End user or business team
Examples	AWS EC2, Azure Virtual Machines	Google App Engine, Azure App Service	Microsoft 365 / Office 365, Gmail, Salesforce

In short: IaaS gives infrastructure, PaaS gives a managed development platform, and SaaS gives finished software.

Task 3: Launch an Ubuntu EC2 Instance

Important: AWS free-tier eligibility, instance availability and pricing depend on the account and current AWS terms. Check the cost estimate and eligibility shown in the console before launching. If the console does not offer t2.micro, select an eligible instance type that meets your instructor's requirements.

1. Sign in to the AWS Management Console and open **EC2**. Choose the required AWS Region.

2. From the EC2 dashboard, select **Instances** → **Launch instances**.

3. Enter a name, such as **College-Fest-Ubuntu**.

4. Under Application and OS Images (AMI), choose **Ubuntu Server 22.04 LTS** (64-bit x86, if available).

5. Choose instance type **t2.micro** if available and eligible for your account.

6. Create or select a key pair. Download the **.pem** file and store it securely; AWS cannot provide the private key again.

7. In Network settings, allow **SSH (TCP 22)** from **My IP** for setup. Avoid opening SSH to everyone (0.0.0.0/0).

8. Review storage and estimated costs, then click **Launch instance**.

9. Return to Instances, wait until Instance state is **Running** and status checks pass. Select the instance to view its Public IPv4 address.

Screenshot evidence to add: Open EC2 → Instances, select the running Ubuntu instance, and take a screenshot showing Instance state = Running and the instance details. Attach your real screenshot here in your submitted assignment.

Screenshot placeholder: _____

Date captured: _____ Instance ID (optional): _____

Task 4: Connect through SSH and Run uname -a

On Windows PowerShell, open the folder containing your downloaded .pem key. Replace the key path and public IP/DNS name with your actual values. The default username for the official Ubuntu AMI is usually **ubuntu**.

```
ssh -i "C:\Users\YOUR_USER\Downloads\your-key.pem" ubuntu@YOUR_PUBLIC_IP
```

If Windows reports that the key permissions are too open, restrict the key file's access in Windows file properties or use an appropriate icacls command for your own Windows username. Never share the .pem key.

After connecting successfully, run:

```
uname -a
```

This command prints the Linux kernel name, hostname, kernel release/version, architecture and other system details.

Record your actual output:

_____

_____

_____

Do not copy a made-up output; paste the output from your own EC2 instance.

Task 5: Host a Static College Fest Website

1. Connect and update: SSH into the Ubuntu EC2 instance and run `sudo apt update`.

2. Install a web server: run `sudo apt install apache2 -y`, then check it with `sudo systemctl enable --now apache2`.

3. Add website files: create or copy your HTML, CSS and image files into `/var/www/html/` (for example, replace or edit `index.html`).

4. Configure access safely: in the EC2 Security Group, allow inbound HTTP (TCP 80) from `0.0.0.0/0` so visitors can reach the public website. Keep SSH (TCP 22) restricted to your own IP. Add HTTPS (TCP 443) and configure TLS if using HTTPS.

5. Test the website: open `http://YOUR_PUBLIC_IP` in a browser. For a domain name, point its DNS record to the server and consider an Elastic IP so the address does not change after a stop/start.

Conclusion

IaaS, PaaS and SaaS provide different levels of cloud management. AWS EC2 is an IaaS service that provides a virtual server. After launching Ubuntu, a user can connect through SSH, verify the system using uname -a, and install a web server to publish a static college-fest website.

Security reminder: Stop or terminate test resources when finished if you no longer need them. Check AWS billing and resource status; stopping an instance may not stop every related charge, such as some storage or public IPv4 charges.